In our toppings example, we call sandwich twice. Each call creates a new instance. So, the sandwich we added toppings to is a different one than the sandwich we're checking for toppings.

The traditional Ruby solution is a common technique called *memoization*, where we store the results of an operation (creating a sandwich) and refer to the stored copy from then on. For more on memoization, see Justin Weiss's article.[4]

A typical Ruby implementation might look something like the following code. Try this new definition for your sandwich method:

01-getting-started/07/spec/sandwich_spec.rb

```
def sandwich
  @sandwich ||= Sandwich.new('delicious', [])
end
```

Now, rerun your specs to make sure that your memoized method fixed the issue.

This pattern is pretty easy to find in Ruby code in the wild, but it's not without its pitfalls. The ||= operator works by seeing if @sandwich is "falsey"—that is, false or nil—before creating a new sandwich. That means it won't work if we're actually trying to store something falsey.

Consider how you'd test a Toaster class that lets you search for a specific toaster by its serial number. If no such toaster exists, the search will return nil. Here's a naïve approach to writing a helper method to cache this value:

01-getting-started/08/toaster.rb

```
def current_toaster
  @current_toaster ||= Toaster.find_by_serial('HHGG42')
end
```

If the search comes up empty, we'll store nil in the @current_toaster variable. On the next call to the helper method, we'll be doing the equivalent of the following code:

01-getting-started/08/toaster.rb

```
@current_toaster = nil || Toaster.find_by_serial('HHGG42')
```

We'll call the potentially slow find_by_serial() method every time; we're not actually memoizing anything. We could concoct a workaround to handle this edge case. But with RSpec, we don't need to.

4. http://www.justinweiss.com/articles/4-simple-memoization-patterns-in-ruby-and-one-gem/